

Lezione 4 - Ottimizzazioni e compilatore

RICERCA BINARIA

◆ Versione con branch

La ricerca binaria è un esempio classico in cui i **branch** sono difficili da prevedere.

👉 il comportamento dipende:

- dai valori nel vettore
- dal valore cercato



Esempio di implementazione:

```
int binary_search(int * v, int len, int key)
{
    int l = 0;
    int r = len;
    while (l < r)
    {
        int m = (l + r) / 2;
        if (v[m] >= key)
            r = m;
        else
            l = m + 1;
    }
    if (v[l] == key)
    {
        return l;
    } else
    {
        return -1;
    }
}
```

👉 ogni iterazione contiene un `if` → quindi un branch



Versione branchless

Possiamo riscrivere lo stesso algoritmo senza branch espliciti:

```
int binary_search_branchless(int * v, int len, int key)
{
    int l = 0;
    int r = len;
    while (l < r)
    {
        int m = (l + r) / 2;
        int q = v[m] >= key;
        r = q*m + (1-q)*r;
        l = q*l + (1-q)*(m+1);
    }
    if (v[l] == key)
    {
        return l;
    } else
    {
        return -1;
    }
}
```

👉 `q` vale:

- 1 se vero
- 0 se falso

👉 quindi si evita il branch

◆ Osservazione importante

Il compilatore può trasformare automaticamente codice con `if` in codice branchless.

👉 per evitare questo comportamento (e confrontare le versioni) si usa:

-fno-if-conversion

👉 funziona con **gcc** (non con clang)

◆ Confronto prestazioni

- per input piccoli → branchless può essere veloce
- per input grandi → **versione con branch può essere migliore**

◆ Perché i branch possono aiutare

Questo è controintuitivo, ma succede per via della CPU moderna.

👉 con i branch:

- la CPU usa **esecuzione speculativa**
- carica dati in anticipo

Esempio:

la CPU può iniziare a leggere:

- $v[(1+m)/2]$
- oppure $v[(m+r)/2]$

prima ancora di sapere quale servirà davvero

👉 se la previsione è corretta:

- il dato è già in cache
- il tempo si riduce

👉 nel branchless:

- non c'è speculazione
- bisogna aspettare il valore corretto

◆ Effetto finale

- branchless → meno hazard di controllo
- con branch → migliore sfruttamento memoria

👉 per array grandi, la memoria diventa il fattore dominante

✳️ quindi i branch possono risultare più veloci

📌 PREFETCH

◆ Problema della memoria

Abbiamo visto che, per array grandi, le prestazioni dipendono molto dalla **memoria**.

👉 accedere alla memoria è lento rispetto alla CPU

👉 quindi:

- se il dato è già in cache → veloce
- se non lo è → bisogna aspettare

◆ Idea del prefetch

Possiamo provare ad anticipare l'accesso alla memoria.

👉 dire al sistema:

“questi dati mi serviranno tra poco”

◆ **__builtin_prefetch**

Il compilatore mette a disposizione una funzione speciale:

```
__builtin_prefetch(void *addr);
```

👉 serve per suggerire alla CPU di caricare un indirizzo in memoria

👉 NON blocca l'esecuzione

👉 è solo un "hint"

◆ **Applicazione alla ricerca binaria**

Nel codice branchless con prefetch:

```
__builtin_prefetch(&v[(l+m)/2]);  
__builtin_prefetch(&v[(m+r)/2]);
```

👉 stiamo dicendo:

- probabilmente useremo uno di questi due indirizzi
- iniziamo a caricarli entrambi

◆ **Effetto**

- i dati vengono caricati prima
- quando servono → sono già disponibili

👉 si riducono i tempi di accesso alla memoria

◆ **Confronto**

- branch → sfrutta esecuzione speculativa
- branchless → può usare prefetch

👉 entrambe cercano di anticipare l'accesso ai dati

◆ Idea generale

Il prefetch serve per:

- ridurre il costo degli accessi alla memoria
- migliorare le prestazioni su dati grandi

✳ è particolarmente utile quando la memoria è il collo di bottiglia

📌 SEQUENZA DI COMPILAZIONE

◆ Preprocessing

Il preprocessing è la prima fase della compilazione.

👉 operazioni principali:

- espansione delle macro
- inclusione degli header (`#include`)
- rimozione dei commenti

Per vedere il risultato:

```
gcc -E source.c
```

👉 stampa il codice preprocessato

◆ Compilazione

In questa fase il compilatore:

- analizza il codice (parsing)
- controlla errori di sintassi

- genera una rappresentazione intermedia
- applica ottimizzazioni
- produce codice assembly

Comando:

```
gcc -S source.c
```

👉 genera un file `source.s`

◆ Assembly

L'assembly viene trasformato in codice macchina.

👉 in questa fase:

- le funzioni esterne non sono ancora risolte
- vengono inseriti dei placeholder

Comando:

```
gcc -c source.c
```

👉 genera un file oggetto `source.o`

◆ Linking

Ultima fase: si crea l'eseguibile.

👉 il linker:

- collega i file `.o`
- risolve i riferimenti alle funzioni

- inserisce il codice necessario

Comando:

```
gcc -o program source1.o source2.o
```

👉 genera il file eseguibile

◆ Idea generale

La compilazione avviene in più fasi:

1. preprocessing
2. compilazione
3. assembly
4. linking

👉 ogni fase ha un ruolo specifico

✳️ capire queste fasi è fondamentale per capire come funziona il codice

📌 ANALISI DEL CODICE GENERATO

◆ Perché analizzare l'assembly

È utile capire quale codice viene generato dal compilatore.

👉 per verificare:

- se le ottimizzazioni che pensiamo di fare sono già applicate
- se il codice generato è quello che ci aspettiamo



Esempio:

```
if (x > 0)
    y = x;
else
    y = -x;
```

- codice con `if` → potrebbe essere trasformato in branchless
- alcune ottimizzazioni potrebbero essere automatiche

◆ Opzione `-fverbose-asm`

Possiamo generare assembly con commenti:

```
gcc -S -fverbose-asm source.c
```

👉 il file `.s` conterrà anche commenti del compilatore

◆ Compiler Explorer

Uno strumento molto utile è:

👉 <https://godbolt.org/>

Permette di:

- vedere l'assembly generato
- confrontare diversi compilatori
- testare ottimizzazioni

◆ Idea importante

Molte ottimizzazioni:

- sono già fatte dal compilatore

- non è necessario implementarle manualmente

👉 quindi è importante controllare il codice generato

👉 è importante verificare se le ottimizzazioni che vogliamo fare manualmente sono già applicate automaticamente dal compilatore

OTTIMIZZAZIONI DEL COMPILATORE

◆ Idea generale

Molte ottimizzazioni vengono fatte automaticamente dal compilatore.

👉 basta attivare l'ottimizzazione (es. -O2 , -O3)

👉 spesso il compilatore:

- semplifica il codice
- elimina operazioni inutili
- migliora le prestazioni

◆ Constant propagation

Se un valore può essere calcolato a compile-time, il compilatore lo sostituisce direttamente.

Esempio:

```
int a = (5 * 6) + 4;  
int b = 2 - 3;  
int c = 2 * 3;  
int d = c / 4;
```

👉 il compilatore può trasformare tutto in valori costanti

◆ Dead code elimination

Il compilatore elimina codice che non influisce sul risultato.



Esempio:

```
int sum = 0;
for (int i = 0; i < 1000; i++)
{
    sum++;
}
printf("finito\n");
```

👉 se `sum` non viene usato, il ciclo può essere eliminato

👉 per evitare che il compilatore elimini il codice:

- si stampa il risultato
- oppure si usa `volatile`

◆ Controllo delle ottimizzazioni

Le ottimizzazioni:

- sono basate su euristiche
- non sempre vengono applicate
- possono essere attivate/disattivate



Esempio:

```
-fno-if-conversion
```

👉 disabilita la trasformazione in branchless

◆ Idea importante

- il compilatore è molto intelligente
- molte ottimizzazioni sono automatiche
- ma dobbiamo capire cosa sta facendo

✳ per questo è importante guardare l'assembly

LOOP UNROLLING

◆ Idea

Il **loop unrolling** consiste nel ridurre il numero di iterazioni di un ciclo ripetendo il corpo più volte.

👉 invece di fare:

```
for (int i = 0; i < 1000; i++)  
{  
    v[i]++;  
}
```

👉 possiamo “srotolare” il ciclo:

```
for (int i = 0; i < 1000; i += 10)  
{  
    v[i]++;  
    v[i+1]++;  
    v[i+2]++;  
    v[i+3]++;  
    v[i+4]++;  
    v[i+5]++;  
    v[i+6]++;  
    v[i+7]++;  
    v[i+8]++;  
    v[i+9]++;  
}
```

◆ Perché funziona

- riduce il numero di branch del ciclo
- riduce l'overhead del controllo (`i++` , confronto, salto)
- migliora il parallelismo

◆ Quando è utile

- cicli con poche operazioni
- cicli molto ripetuti
- quando il numero di iterazioni è noto

◆ Ottimizzazione del compilatore

Il compilatore può fare loop unrolling automaticamente.

👉 opzioni:

`-funroll-loops`

`-fno-unroll-loops`

👉 ma:

- non è garantito
- dipende da euristiche

◆ Forzare il comportamento

Possiamo forzare il compilatore con:

```
#pragma GCC unroll 16
```

👉 forza lo “srotolamento” del ciclo 16 volte

◆ Collegamento con il codice

Esempio reale:

👉 mostra un ciclo semplice che può essere ottimizzato

◆ Attenzione

- aumenta la dimensione del codice
- non sempre migliora le prestazioni

👉 è un compromesso tra:

- velocità
- uso della memoria

◆ Idea generale

Il loop unrolling serve per:

- ridurre il costo dei cicli
- migliorare le prestazioni

✳️ ma va usato con attenzione

📌 FUNCTION INLINING

◆ Idea

Il **function inlining** consiste nel sostituire una chiamata a funzione con il corpo della funzione stessa.



Esempio:

```
inline int f(int a, int b)
{
    int s = 0;
    while (a < b)
    {
        s += a;
        a++;
    }
    return s;
}
```

Quando viene chiamata:

```
int x = f(10, 20);
```

👉 invece di chiamare la funzione, il compilatore può inserire direttamente il codice al posto della chiamata

◆ Perché funziona

Evita il costo della chiamata a funzione:

- salvataggio dei registri
- gestione dello stack
- salto alla funzione

👉 quindi:

- codice più veloce
- meno overhead

◆ Svantaggi

- il codice viene duplicato
- aumenta la dimensione del programma

◆ Ruolo del compilatore

Il compilatore può decidere automaticamente se fare inlining.

👉 ma possiamo suggerirlo con:

```
inline int f(int a, int b)
```

◆ Collegamento con il codice

Esempio reale:

👉 confronto tra funzione `f` (inline) e `g` (normale)

◆ Idea generale

Il function inlining serve per:

- ridurre il costo delle chiamate a funzione
- migliorare le prestazioni

✳️ ma aumenta la dimensione del codice

📌 BRANCH LIKELINESS

◆ Idea

Se sappiamo che un branch è più probabile di un altro, possiamo dirlo al compilatore.

👉 questo aiuta il compilatore a fare ottimizzazioni migliori

◆ **__builtin_expect**

GCC fornisce una funzione speciale:

```
__builtin_expect(expr, value)
```

👉 dove:

- `expr` → espressione (es. `i < n`)
- `value` → valore atteso (1 vero, 0 falso)



Esempio:

```
if (__builtin_expect(v[i] < threshold, 0))  
{  
    sum += v[i];  
}
```

👉 stiamo dicendo:

- questa condizione è probabilmente falsa

◆ **Effetto**

Il compilatore può:

- organizzare meglio il codice
- migliorare la predizione dei branch
- ridurre il costo delle misprediction

◆ **Collegamento con il codice**

Esempio reale:

👉 mostra come varia il tempo al variare della probabilità del branch

◆ **Idea importante**

- non cambia il comportamento del programma
- è solo un suggerimento per il compilatore

◆ **Quando usarlo**

- quando conosciamo la probabilità di un branch
- codice critico per le prestazioni

◆ **Idea finale**

Possiamo riassumere le ottimizzazioni così:

- compilatore → ottimizza automaticamente
- noi → possiamo dare suggerimenti